

Module 1: Random Number Generation and Simulation Foundations

MA 563 – Statistical Computation and Simulation

Dr. Rishikesh Yadav

`rishikesh@iitmandi.ac.in` — TA: Vedant Vibhor (`D24228@students.iitmandi.ac.in`)

School of Mathematics and Statistical Sciences, IIT Mandi

Table of Contents

Pseudo-Random Number Generators

Motivation and Desired Properties

Linear Congruential Generators (LCG)

Modern Generators

Generating Uniform and Non-Uniform Random Variables

From PRNG Output to $U(0, 1)$

Strategies for Non-Uniform Generation

Inverse Transform Method

Theory: The Probability Integral Transform

Algorithm and Continuous Examples

The Discrete Case

Limitations

Acceptance–Rejection Method

Setup and Theory

Proof of Correctness and Efficiency

Worked Examples

Box–Muller Transformation

Derivation

Basic Algorithm

Ratio-of-Uniforms Method

Module 1: Random Number Generation and Simulation Foundations (8 Lectures)

Pseudo-random number generators, uniform and non-uniform random variable generation, inverse transform method, acceptance–rejection method, Box–Muller transformation, ratio-of-uniforms method, diagnosing randomness, and simulation error.

Why start here?

Every simulation-based method in this course – Monte Carlo integration, bootstrap, EM, MCMC – assumes you can already generate reliable random numbers from arbitrary distributions. Module 1 builds that foundation from first principles.

Pseudo-Random Number Generators

- Motivation and Desired Properties

- Linear Congruential Generators (LCG)

- Modern Generators

Generating Uniform and Non-Uniform Random Variables

- Inverse Transform Method

- Acceptance–Rejection Method

- Box–Muller Transformation

- Ratio-of-Uniforms Method

- Diagnosing Randomness

- Unit Summary

Why “Pseudo”-Random?

- Computers are deterministic machines: given the same input, they always produce the same output.
- True randomness would need a physical source (radioactive decay, thermal noise, atmospheric noise). This is slow, non-reproducible, and impractical for billions of simulation draws.
- Instead, we use a **deterministic algorithm** that produces a sequence of numbers *statistically indistinguishable* from i.i.d. $\text{Uniform}(0, 1)$ draws.

Definition: Pseudo-Random Number Generator (PRNG)

A PRNG is a deterministic algorithm that, starting from an initial value called the **seed** X_0 , produces a sequence $X_1, X_2, X_3, \dots$ that mimics the statistical behaviour of i.i.d. uniform random variables.

Desired Properties of a Good PRNG

- **Uniformity:** the output values should be spread evenly over $(0, 1)$.
- **Independence:** successive values should show no detectable correlation or pattern.
- **Long period:** the sequence should not repeat for an extremely long time (period $\gg$ number of draws needed).
- **Reproducibility:** the same seed must always regenerate the exact same sequence (essential for debugging and replicable research).
- **Efficiency:** generation must be fast – simulations often require billions of draws.
- **Portability:** the same seed should give the same sequence across machines/languages when required.

The Linear Congruential Generator (LCG)

Recurrence

$$X_{n+1} = (aX_n + c) \bmod m, \quad U_n = \frac{X_n}{m}$$

- m – the **modulus** ($m > 0$)
- a – the **multiplier** ($0 \leq a < m$)
- c – the **increment** ($0 \leq c < m$)
- X_0 – the **seed** ($0 \leq X_0 < m$)
- If $c = 0$: called a **multiplicative congruential generator (MCG)**.
- If $c \neq 0$: called a **mixed congruential generator**.
- The sequence $\{X_n\}$ is purely deterministic once (a, c, m, X_0) are fixed – but $\{U_n\}$ can pass strong statistical tests for uniformity and independence.

Full-Period Conditions: The Hull–Dobell Theorem

Classic Example: Lehmer / Park–Miller Generator

$$m = 2^{31} - 1 = 2147483647 \text{ (a Mersenne prime),} \quad a = 7^5 = 16807, \quad c = 0$$

This multiplicative generator has period $m - 1$ and was, for decades, the default generator in many statistical packages.

Theorem (Hull–Dobell, 1962)

An LCG has full period m (i.e., it cycles through all integers $0, \dots, m - 1$ before repeating) if and only if:

1. $\gcd(c, m) = 1$ (c and m are coprime);
2. $a - 1$ is divisible by every prime factor of m ;
3. $a - 1$ is divisible by 4, if m is divisible by 4.

Note: Note that the Park-Miller generator is not an example of Hull-Dobell Theorem

Worked Numerical Example

Take a tiny toy LCG: $m = 16$, $a = 5$, $c = 3$, $X_0 = 7$.

$$X_{n+1} = (5X_n + 3) \bmod 16$$

n	0	1	2	3	4	5
X_n	7	6	1	8	11	10
$U_n = X_n/16$	0.4375	0.375	0.0625	0.500	0.6875	0.625

- Check Hull–Dobell: $\gcd(3, 16) = 1$ ✓; prime factor of 16 is 2, and $a - 1 = 4$ divisible by 2 ✓; m divisible by 4 and $a - 1 = 4$ divisible by 4 ✓ $\Rightarrow$ full period 16.
- In practice m is astronomically large (e.g. $2^{31} - 1$ or bigger), so the period is enormous.

The Lattice Structure Problem

Marsaglia's Theorem (1968) – “Successive random numbers generated by an LCG do not fill space uniformly.”

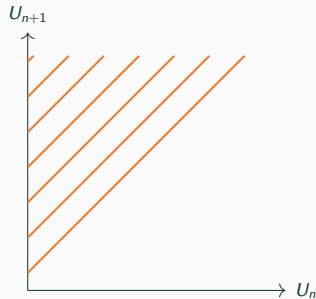
Overlapping k -tuples $(U_n, U_{n+1}, \dots, U_{n+k-1})$ generated by an LCG do *not* fill the k -dimensional unit cube uniformly – they lie on a limited number of parallel hyperplanes. **We will see this in Lab**

The Lattice Structure Problem

Marsaglia's Theorem (1968) – “Successive random numbers generated by an LCG do not fill space uniformly.”

Overlapping k -tuples $(U_n, U_{n+1}, \dots, U_{n+k-1})$ generated by an LCG do *not* fill the k -dimensional unit cube uniformly – they lie on a limited number of parallel hyperplanes. **We will see this in Lab**

- This is a structural flaw of *all* LCGs.
- It can bias simulations that rely on multivariate uniform draws (e.g., Monte Carlo integration in several dimensions).



Points cluster on parallel lines/planes

Beyond the LCG: Modern PRNGs

- **Mersenne Twister (MT19937):** default generator in R, Python, MATLAB, NumPy.
 - Astronomically long period $2^{19937} - 1$.
 - Passes almost all standard statistical test suites.
 - Unlike LCGn, which relies on just the previous number ($X - n$), Mersenne Twister relies on the previous 624 numbers. $\Rightarrow$ **Patterns almost disappear**
 - Not cryptographically secure (state can be reconstructed from enough outputs), but excellent for simulation.
- **Xorshift / xoshiro family:** extremely fast, use bitwise XOR and shift operations; good statistical quality with tiny state.
- **PCG (Permuted Congruential Generator):** combines an LCG core with a permutation output function – small state, excellent statistical properties.
- **Cryptographic RNGs** (e.g., ChaCha20, hardware TRNGs): needed when unpredictability itself matters (security, gambling), not just statistical quality.

Note: For this course, think of a modern PRNG (e.g. Mersenne Twister) as a “black box” that reliably delivers a stream $U_1, U_2, U_3, \dots \overset{\text{approx}}{\sim}$ i.i.d. $\text{Unif}(0, 1)$. Everything else in this unit is about *transforming* that stream into other distributions.

Pseudo-Random Number Generators

Generating Uniform and Non-Uniform Random Variables

From PRNG Output to $U(0, 1)$

Strategies for Non-Uniform Generation

Inverse Transform Method

Acceptance–Rejection Method

Box–Muller Transformation

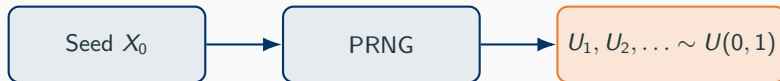
Ratio-of-Uniforms Method

Diagnosing Randomness

Unit Summary

The Building Block: $\text{Uniform}(0, 1)$

- A PRNG's raw output is an integer $X_n \in \{0, 1, \dots, m - 1\}$.
- Dividing by m (or $m - 1$, depending on convention) rescales it to $(0, 1)$: $U_n = X_n/m$.
- This stream of (approximately) i.i.d. $U(0, 1)$ variables is the **universal raw material** from which every other distribution in this course is built.



From Uniform to Everything Else: A Roadmap

Given a stream of $U(0, 1)$ variables, how do we generate $X \sim F$ for an *arbitrary* target distribution F ? Four general strategies, all covered in this module:

1. **Inverse Transform Method** – invert the target CDF directly: $X = F^{-1}(U)$.
2. **Transformation Methods** – express X as a deterministic function of one or more uniforms/other random variables with known distributions (e.g. Box–Muller for the Normal).
3. **Acceptance–Rejection Method** – sample from an easy “envelope” distribution and probabilistically accept/reject to correct the shape.
4. **Ratio-of-Uniforms Method** – generate a pair of uniforms over a cleverly chosen 2-D region whose ratio has the target distribution.

Why so many methods?

No single method is best for every distribution: some have closed-form inverse CDFs (easy), some don't (e.g. Normal), some are only known up to a normalizing constant (common in Bayesian statistics – foreshadowing Module 5!).

Pseudo-Random Number Generators

Generating Uniform and Non-Uniform Random Variables

Inverse Transform Method

Theory: The Probability Integral Transform

Algorithm and Continuous Examples

The Discrete Case

Limitations

Acceptance–Rejection Method

Box–Muller Transformation

Ratio-of-Uniforms Method

Diagnosing Randomness

Unit Summary

The Key Theorem

Theorem (Probability Integral Transform)

Let F be a continuous, strictly increasing CDF, and let $U \sim \text{Uniform}(0, 1)$. Define

$$X = F^{-1}(U).$$

Then X has CDF F .

Proof

$$\begin{aligned} P(X \leq x) &= P(F^{-1}(U) \leq x) \\ &= P(U \leq F(x)) \quad (\text{since } F \text{ is monotonic increasing}) \\ &= F(x) \quad (\text{since } U \sim \text{Unif}(0, 1), \text{ so } P(U \leq u) = u). \end{aligned}$$

This is the theoretical engine behind the **inverse transform method**: if we can invert F , we can sample from it.

Inverse Transform Algorithm

Algorithm: Inverse Transform Sampling

1. Generate $U \sim \text{Uniform}(0, 1)$.
2. Compute and return $X = F^{-1}(U)$.

Example 1: Exponential Distribution

$F(x) = 1 - e^{-\lambda x}$, $x \geq 0$. Solve $u = 1 - e^{-\lambda x}$ for x :

$$X = F^{-1}(U) = -\frac{\ln(1 - U)}{\lambda}.$$

Since $U \sim \text{Unif}(0, 1) \Rightarrow 1 - U \sim \text{Unif}(0, 1)$ too, we may equivalently use

$$X = -\frac{\ln U}{\lambda}.$$

More Continuous Examples

Example 2: Standard Cauchy Distribution

$F(x) = \frac{1}{2} + \frac{1}{\pi} \arctan(x)$. Inverting:

$$X = \tan\left(\pi\left(U - \frac{1}{2}\right)\right).$$

Example 3: Weibull Distribution

$F(x) = 1 - e^{-(x/\beta)^\alpha}$. Inverting:

$$X = \beta(-\ln(1 - U))^{1/\alpha}.$$

Example 4: Pareto Distribution

$F(x) = 1 - (x_m/x)^\alpha$, $x \geq x_m$. Inverting:

$$X = \frac{x_m}{(1 - U)^{1/\alpha}}.$$

Inverse Transform for Discrete Random Variables

For a discrete r.v. taking values $x_1 < x_2 < \dots$ with $P(X = x_i) = p_i$, define the CDF as a step function. The algorithm partitions $(0, 1)$ into consecutive intervals of length $p_1, p_2, \dots$

Algorithm

1. Generate $U \sim \text{Unif}(0, 1)$.
2. Find the smallest i such that $U \leq \sum_{j \leq i} p_j = F(x_i)$.
3. Return $X = x_i$.

Example: Biased Coin / Categorical Variable

Suppose $P(X=1)=0.2$, $P(X=2)=0.5$, $P(X=3)=0.3$. Then

$$X = \begin{cases} 1 & \text{if } U \in [0, 0.2) \\ 2 & \text{if } U \in [0.2, 0.7) \\ 3 & \text{if } U \in [0.7, 1) \end{cases}$$

A single uniform draw and a lookup table generates the categorical variable.

When the Inverse Transform Method Struggles

- Requires F^{-1} in **closed form** – not always available.
- **Classic failure case: the Normal distribution.** Its CDF $\Phi(x)$ has no elementary closed-form inverse.
 - Workaround 1: numerical approximation of Φ^{-1} (e.g. rational approximations, Newton–Raphson root finding on $\Phi(x) - u = 0$).
 - Workaround 2: use a specialized transformation method instead → **Box–Muller** (coming up).
- For multivariate distributions, F^{-1} is rarely tractable at all.
- Even when F^{-1} exists, it can be numerically unstable near $U \approx 0$ or $U \approx 1$ (extreme tails).

Motivates the next methods

When F^{-1} is unavailable or expensive, we turn to **acceptance–rejection**, **special transformations** (Box–Muller), and the **ratio-of-uniforms** method.

Pseudo-Random Number Generators

Generating Uniform and Non-Uniform Random Variables

Inverse Transform Method

Acceptance–Rejection Method

- Setup and Theory

- Proof of Correctness and Efficiency

- Worked Examples

Box–Muller Transformation

Ratio-of-Uniforms Method

Diagnosing Randomness

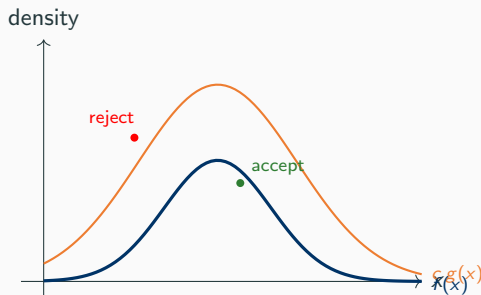
Unit Summary

The Idea: Darts Under a Curve

- Target density $f(x)$, hard to sample directly.
- Choose an **envelope/proposal** density $g(x)$ that is easy to sample from.
- Choose a constant $c \geq 1$ such that

$$f(x) \leq c g(x) \quad \text{for all } x.$$

- **Idea:** sample uniformly “under” the curve $c g(x)$; keep only the points that also fall under $f(x)$.



Algorithm: Acceptance–Rejection Sampling

1. Generate $Y \sim g$.
2. Generate $U \sim \text{Uniform}(0, 1)$, independent of Y .
3. If

$$U \leq \frac{f(Y)}{c g(Y)},$$

accept $X = Y$. Otherwise **reject** and go to Step 1.

- Note c must satisfy $c = \sup_x \frac{f(x)}{g(x)}$ (or any valid upper bound) for the ratio in Step 3 to always lie in $[0, 1]$.
- Works even when f is known only up to a normalizing constant – a property that becomes crucial later, in Bayesian computation (Module 5).

Why Does This Work?

Proof sketch

Condition on acceptance. The joint “proposal” density of (Y, U) is $g(y)$ on y times uniform on $[0, 1]$ for u . The accepted region is $\{(y, u) : 0 \leq u \leq f(y)/(c g(y))\}$.

$$P(Y \leq x \mid \text{accept}) = \frac{\int_{-\infty}^x g(y) \cdot \frac{f(y)}{c g(y)} dy}{\int_{-\infty}^{\infty} g(y) \cdot \frac{f(y)}{c g(y)} dy} = \frac{\int_{-\infty}^x f(y) dy}{\int_{-\infty}^{\infty} f(y) dy} = F(x).$$

So the *accepted* Y values have exactly the target CDF F . ■

Efficiency of Acceptance–Rejection

- Each proposal Y is accepted with probability $P(\text{accept}) = \frac{1}{c}$.
- **Design goal:** choose g as close in shape to f as possible, so that $c = \sup f/g$ is close to 1 $\Rightarrow$ high acceptance rate, few wasted draws.
- The number of proposals needed until one acceptance follows a **Geometric distribution** with mean c .

Trade-off

A very tight envelope ($c \approx 1$) gives high efficiency but may be harder to construct or sample from; a loose, simple envelope is easy to sample but wastes many draws (small acceptance probability).

Example: Sampling from a Beta(2, 2) Density

Target: $f(x) = 6x(1 - x)$, $0 \leq x \leq 1$.

Take **envelope** $g(x) = 1$ (i.e., $\text{Uniform}(0, 1)$), the simplest possible proposal.

$$c = \sup_{0 \leq x \leq 1} \frac{f(x)}{g(x)} = \sup_x 6x(1 - x) = 6 \cdot \frac{1}{2} \cdot \frac{1}{2} = 1.5 \quad (\text{at } x = 0.5).$$

Algorithm

1. Generate $Y \sim \text{Unif}(0, 1)$.
2. Generate $U \sim \text{Unif}(0, 1)$.
3. Accept $X = Y$ if $U \leq \frac{6Y(1 - Y)}{1.5} = 4Y(1 - Y)$; else repeat.

Expected number of proposals per acceptance = $c = 1.5$ – quite efficient.

Example: Sampling from a Truncated Normal

Goal: sample $X \sim N(0, 1)$ truncated to $[a, \infty)$ for large $a > 0$ (relevant e.g. for rare-event / tail simulation).

- Naive rejection (sample $N(0, 1)$, keep only $X \geq a$) is extremely inefficient for large a – acceptance probability = $P(Z \geq a)$, which is tiny.
- Better proposal: a shifted **Exponential** density on $[a, \infty)$,

$$g(x) = a e^{-a(x-a)}, \quad x \geq a,$$

chosen because it decays at a similar rate to the Normal tail.

- Compute the ratio $f(x)/g(x)$ and its supremum to get c ; acceptance probability is then far higher than naive rejection.

Lesson

The choice of envelope g is where the “art” of acceptance–rejection lies – match tail behaviour to the target for high efficiency.

Pseudo-Random Number Generators

Generating Uniform and Non-Uniform Random Variables

Inverse Transform Method

Acceptance–Rejection Method

Box–Muller Transformation

Derivation

Basic Algorithm

Ratio-of-Uniforms Method

Diagnosing Randomness

Unit Summary

Why Normal Variables Need a Special Trick

- The Normal CDF $\Phi(x)$ has **no closed-form inverse** $\Rightarrow$ inverse transform method is not directly usable (without numerical approximation).
- **Box–Muller (1958)** sidesteps this entirely using a clever change of variables based on **polar coordinates**.

Key Idea

Consider two independent standard normal variables (Z_1, Z_2) . Their joint density is rotationally symmetric:

$$f(z_1, z_2) = \frac{1}{2\pi} e^{-(z_1^2 + z_2^2)/2}.$$

Switch to polar coordinates R, Θ where $Z_1 = R \cos \Theta$, $Z_2 = R \sin \Theta$. One can show:

- $R^2 = Z_1^2 + Z_2^2 \sim \text{Exponential}(1/2)$ (equivalently χ_2^2),
- $\Theta \sim \text{Uniform}(0, 2\pi)$, independent of R .

From Polar Back to Cartesian

Since $R^2 \sim \text{Exponential}(1/2)$, by the inverse transform method (Section 3!):

$$R^2 = -2 \ln U_1 \implies R = \sqrt{-2 \ln U_1}, \quad U_1 \sim \text{Unif}(0, 1).$$

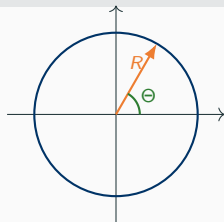
And since $\Theta \sim \text{Unif}(0, 2\pi)$:

$$\Theta = 2\pi U_2, \quad U_2 \sim \text{Unif}(0, 1), \text{ independent of } U_1.$$

Box-Muller Transformation

$$Z_1 = \sqrt{-2 \ln U_1} \cos(2\pi U_2), \quad Z_2 = \sqrt{-2 \ln U_1} \sin(2\pi U_2)$$

gives two **independent** standard Normal variables from two independent uniforms.



Box–Muller: Algorithm

Algorithm: Basic Box–Muller

1. Generate $U_1, U_2 \stackrel{\text{iid}}{\sim} \text{Uniform}(0, 1)$.

2. Compute

$$Z_1 = \sqrt{-2 \ln U_1} \cos(2\pi U_2), \quad Z_2 = \sqrt{-2 \ln U_1} \sin(2\pi U_2).$$

3. Return Z_1, Z_2 (a free “bonus” second draw at no extra cost!).

General Normal $N(\mu, \sigma^2)$

$$X = \mu + \sigma Z$$

for Z generated above – a simple location-scale shift.

Practical note

Computing $\cos$, $\sin$, and $\ln$ is relatively expensive. This motivates a trig-free variant next.

Pseudo-Random Number Generators

Generating Uniform and Non-Uniform Random Variables

Inverse Transform Method

Acceptance–Rejection Method

Box–Muller Transformation

Ratio-of-Uniforms Method

Theory

Practical Algorithm

Example: Standard Normal via Ratio-of-Uniforms

Diagnosing Randomness

Unit Summary

The Ratio-of-Uniforms Idea

Theorem (Kinderman–Monahan, 1977)

Let $h(x) \geq 0$ be proportional to the target density $f(x)$ (i.e. $f(x) = h(x) / \int h$). Define the planar region

$$C = \left\{ (u, v) : x = \frac{v}{u}, 0 \leq u \leq \sqrt{h(x)}, b^- \leq v \leq b^+ \right\}.$$

If (U, V) is drawn **uniformly** over C , then $X = V/U$ has density f .

- Beautiful because it needs f only up to a normalizing constant – exactly like acceptance–rejection.
- Sampling uniformly over the (often irregularly shaped) region C is done by enclosing C in a bounding rectangle and rejecting points that fall outside C .

Constructing the Bounding Rectangle

For a density with a bounded, unimodal $h(x)$, define

$$a = \sup_x \sqrt{h(x)}, \quad b^+ = \sup_x x\sqrt{h(x)}, \quad b^- = \inf_x x\sqrt{h(x)}.$$

Then $C \subset [0, a] \times [b^-, b^+]$, a simple bounding rectangle.

Algorithm: Ratio-of-Uniforms Sampling

1. Generate $U \sim \text{Uniform}(0, a)$ and $V \sim \text{Uniform}(b^-, b^+)$.
2. Compute $X = V/U$.
3. **Accept** X if $U \leq \sqrt{h(X)}$; otherwise **reject** and return to Step 1.

Worked Example: Standard Normal

Take $h(x) = e^{-x^2/2}$ (proportional to the standard Normal density).

$$a = \sup_x \sqrt{h(x)} = \sup_x e^{-x^2/4} = 1 \quad (\text{at } x = 0),$$

$$b^+ = \sup_x x e^{-x^2/4}. \quad \text{Setting the derivative to zero:}$$

$$\frac{d}{dx} [x e^{-x^2/4}] = e^{-x^2/4} \left(1 - \frac{x^2}{2}\right) = 0 \Rightarrow x = \sqrt{2},$$

$$b^+ = \sqrt{2} e^{-1/2} \approx 0.8578, \quad b^- = -b^+ \text{ (by symmetry).}$$

Resulting Algorithm

1. $U \sim \text{Unif}(0, 1)$, $V \sim \text{Unif}(-0.8578, 0.8578)$.
2. $X = V/U$.
3. Accept if $U \leq \exp(-X^2/4)$, i.e. $-4 \ln U \geq X^2$; else repeat.

Note: No logarithm of a ratio, no trigonometry – just algebra and one exponential/log evaluation

Ratio-of-Uniforms: When to Use It

- Attractive when the density is known only up to a constant (common in Bayesian posteriors – more in Module 5).
- Often more efficient than naive acceptance–rejection because the bounding region can be tailored tightly to h .
- Generalizes naturally to some multivariate settings.
- Like acceptance–rejection, its efficiency (acceptance rate) is governed by how tightly the rectangle bounds C – the ratio $\frac{\text{Area}(C)}{\text{Area of rectangle}}$.

Pseudo-Random Number Generators

Generating Uniform and Non-Uniform Random Variables

Inverse Transform Method

Acceptance–Rejection Method

Box–Muller Transformation

Ratio-of-Uniforms Method

Diagnosing Randomness

Uniformity Tests

Independence and Pattern Tests

Visual Diagnostics

Unit Summary

Why We Must Test Our Random Numbers

- A PRNG is a deterministic algorithm – it is not *obvious* a priori that its output behaves like true i.i.d. uniforms.
- Flawed generators have caused real, documented simulation failures (e.g. the infamous RANDU generator of the 1960s–70s, which had severe 3-D lattice structure).
- We need formal statistical tests, applied *before* trusting a generator (or a specific implementation/seed) for serious work.

Testing Uniformity

Chi-Square Goodness-of-Fit Test

Partition $(0, 1)$ into k equal bins. Let O_i = observed count in bin i out of n draws, and $E_i = n/k$ = expected count under uniformity.

$$\chi^2 = \sum_{i=1}^k \frac{(O_i - E_i)^2}{E_i} \underset{\sim}{\text{approx}} \chi_{k-1}^2 \text{ under } H_0.$$

Reject uniformity if χ^2 exceeds the critical value.

Kolmogorov–Smirnov (KS) Test

Compare the empirical CDF $F_n(x)$ of the sample to the target CDF $F(x) = x$ on $(0, 1)$:

$$D_n = \sup_x |F_n(x) - F(x)|.$$

Large D_n (relative to its known sampling distribution) indicates a departure from uniformity.

Testing Independence / Detecting Patterns

- **Serial correlation test:** Compute the sample autocorrelation

$$\hat{\rho}_k = \frac{\sum_{n=1}^{N-k} (U_n - \bar{U})(U_{n+k} - \bar{U})}{\sum_{n=1}^N (U_n - \bar{U})^2}$$

at several lags k .

- For independent U_n , $\rho_k = 0$ for every $k > 0$.
- Hence, $\hat{\rho}_k$ should fluctuate randomly around 0.
- Significant nonzero values indicate possible dependence between successive outputs.
- **Runs test:** Convert the sequence into a binary sequence, e.g.,

$$U_n < 0.5 \Rightarrow 0, \quad U_n \geq 0.5 \Rightarrow 1.$$

A *run* is a consecutive sequence of identical values.

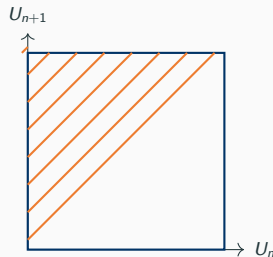
- Under independence, the number of runs has a known expected value.
- Too few runs $\Rightarrow$ outputs tend to cluster; too many runs $\Rightarrow$ excessive alternation.
- **Spectral test:** Examine successive tuples in k dimensions

$$(U_n, U_{n+1}, \dots, U_{n+k-1})$$

- Ideally, these points should fill the k -dimensional unit cube uniformly.
- Poor generators may produce points lying approximately on a small number of parallel hyperplanes—a **lattice structure**.

Visual Diagnostics

- **Histogram** of U_n : should look flat over $(0, 1)$.
- **Scatterplot of pairs** (U_n, U_{n+1}) : should fill the unit square with no visible structure.
- **QQ-plot** against the theoretical uniform quantiles: points should lie on the 45° line.



A flawed LCG: visible lattice lines

A picture can reveal what a single test statistic misses

The infamous RANDU generator passed some 1-D tests but showed obvious planar structure in a 3-D scatterplot – a cautionary tale for always checking visually *and* formally.

Practical takeaway

You will rarely write your own PRNG in practice – but you should know *how* to check whether the one you're relying on (R's, Python's, etc.) is trustworthy for your specific application, especially in high dimensions or for rare-event simulation.

Pseudo-Random Number Generators

Generating Uniform and Non-Uniform Random Variables

Inverse Transform Method

Acceptance–Rejection Method

Box–Muller Transformation

Ratio-of-Uniforms Method

Diagnosing Randomness

Unit Summary

Putting It All Together

Method	Needs	Best suited for
Inverse Transform	Closed-form F^{-1}	Exponential, Weibull, Pareto, discrete r.v.s
Acceptance–Rejection	Envelope g , bound c	General shapes; f known up to constant
Box–Muller	—	Normal (exact, elegant)
Ratio-of-Uniforms	$h \propto f$	Unimodal densities; f known up to constant

The Big Picture

- A trustworthy **PRNG** gives us the raw stream of $U(0, 1)$ variables.
- **Transformation techniques** turn that stream into draws from any target distribution.
- We must always **diagnose randomness** (formally and visually) and be honest about **simulation error**.

Next: Module 2 – Monte Carlo Methods